

Relational Algebra

- ☆ It is a **procedural** query language.
- ☆ RA is a **set of operations** on relation(s).
- ☆ It is a set of **algebraic operations**.
- ☆ Input : One or more relations.
- ☆ Output : A relation.
- ☆ It provides a **theoretical foundation** for relational databases.
- ☆ **Structured Query Language (SQL)**.
- ☆ It allows us to understand database operations in more detail and motivate us to write **optimized queries**.

1

Relational Algebra Operations

Fundamental operations:

- ☆ Select (σ)
- ☆ Project (Π)
- ☆ Union ($\cup$)
- ☆ Set Difference ($-$)
- ☆ Cartesian Product ($\times$)
- ☆ Rename (ρ)

Relational Algebra Operations

Additional operations:

- ☆ Set intersection ($\cap$)
- ☆ Assignment operation (=)
- ☆ Natural join ($\bowtie_*$)
- ☆ Division operation ($\div$)
- ☆ Outer join
 - Left outer join ($\bowtie\!\!\!\Join$)
 - Right outer join ($\Join\!\!\!\bowtie$)
 - Full outer join ($\bowtie\!\!\!\Join\!\!\!\bowtie$)

1. Select (σ)

- ☆ Select tuples that satisfy a given predicate.
- ☆ It is denoted by lowercase Greek letter sigma (σ).
- ☆ Syntax: $\sigma_{\text{<selection_condition>}}$ (Relation).
- ☆ Example: $\sigma_{D_ID = 2}$ (Employee).
- ☆ Comparison operators: $=$, $\neq$, $<$, $\leq$, $>$, and $\geq$.
- ☆ Connectives: AND ($\wedge$), OR ($\vee$) and NOT ($\neg$).

1. Select (σ)

Example 1: Write an RA expression to find all the instructors working in "Finance" department.

Solution:

$\sigma_{\text{Dept_Name} = \text{"Finance"}}(\text{INSTRUCTOR})$

Output:

ID	Name	Dept_Name	Salary
26589	Yusuf	Finance	95000
12547	Neil	Finance	80000

INSTRUCTOR			
ID	Name	Dept_Name	Salary
10101	John	Biology	65000
12121	Robin	Computer Science	90000
25252	Alya	Electrical	40000
26589	Yusuf	Finance	95000
54789	Ravi	Music	60000
78787	Raj	Physics	87000
87458	Jayant	History	75000
76985	Pratik	Computer Science	89000
12547	Neil	Finance	80000

1. Select (σ)

Example 2: Find all instructors with salary greater than \$87,000.

Solution:

$\sigma_{\text{Salary} > 87000}(\text{INSTRUCTOR})$

Output:

ID	Name	Dept_Name	Salary
12121	Robin	Computer Science	90000
26589	Yusuf	Finance	95000
76985	Pratik	Computer Science	89000

INSTRUCTOR			
ID	Name	Dept_Name	Salary
10101	John	Biology	65000
12121	Robin	Computer Science	90000
25252	Alya	Electrical	40000
26589	Yusuf	Finance	95000
54789	Ravi	Music	60000
78787	Raj	Physics	87000
87458	Jayant	History	75000
76985	Pratik	Computer Science	89000
12547	Neil	Finance	80000

1. Select (σ)

Example 3: Find all instructors who are working in "Finance" department and drawing the salary greater than \$87,000.

Solution:

$\sigma_{\text{Dept_Name} = \text{"Finance"} \wedge \text{Salary} > 87000}(\text{INSTRUCTOR})$

Output:

ID	Name	Dept_Name	Salary
26589	Yusuf	Finance	95000

INSTRUCTOR			
ID	Name	Dept_Name	Salary
10101	John	Biology	65000
12121	Robin	Computer Science	90000
25252	Alya	Electrical	40000
26589	Yusuf	Finance	95000
54789	Ravi	Music	60000
78787	Raj	Physics	87000
87458	Jayant	History	75000
76985	Pratik	Computer Science	89000
12547	Neil	Finance	80000

Homework

Question 1: Write an RA expression to find all the instructors drawing salary greater than 60000.

Question 2: Write an RA expression to find all the instructors drawing salary between 50000 and 75000.

Question 3: Write an RA expression to find all the instructors in the INSTRUCTOR relation.

INSTRUCTOR			
ID	Name	Dept_Name	Salary
10101	John	Biology	65000
12121	Robin	Computer Science	90000
25252	Alya	Electrical	40000
26589	Yusuf	Finance	95000
54789	Ravi	Music	60000
78787	Raj	Physics	87000
87458	Jayant	History	75000
76985	Pratik	Computer Science	89000
12547	Neil	Finance	80000

2. Project (Π)

- ☆ It returns its argument relation with **certain attributes left out**.
- ☆ It is a **unary** operator.
- ☆ It is denoted by the uppercase Greek letter **pi** (Π).
- ☆ Basically a relation is a **set**.
- ☆ In the result, the **duplicate rows** are eliminated.
- ☆ Syntax: Π **Attribute1, Attribute2, ... (Relation)**.

2. Project (Π)

Example 1: List all instructors' ID, name, and salary, but do not care about the dept_name.

Solution:

Π ID, Name, Salary (INSTRUCTOR)

Output:

ID	Name	Salary
10101	John	65000
12121	Robin	90000
25252	Alya	40000
26589	Yusuf	95000
54789	Ravi	60000
78787	Raj	87000
87458	Jayant	75000
76985	Pratik	89000
12547	Neil	80000

INSTRUCTOR

ID	Name	Dept_Name	Salary
10101	John	Biology	65000
12121	Robin	Computer Science	90000
25252	Alya	Electrical	40000
26589	Yusuf	Finance	95000
54789	Ravi	Music	60000
78787	Raj	Physics	87000
87458	Jayant	History	75000
76985	Pratik	Computer Science	89000
12547	Neil	Finance	80000

2. Project (Π)

Example 2: Find the name of all instructors in the Computer Science department.

Solution:

$\Pi_{\text{Name}} (\sigma_{\text{Dept_Name} = \text{"Computer Science"}} (\text{INSTRUCTOR}))$

Output:

Name
Robin
Pratik

INSTRUCTOR			
ID	Name	Dept_Name	Salary
10101	John	Biology	65000
12121	Robin	Computer Science	90000
25252	Alya	Electrical	40000
26589	Yusuf	Finance	95000
54789	Ravi	Music	60000
78787	Raj	Physics	87000
87458	Jayant	History	75000
76985	Pratik	Computer Science	89000
12547	Neil	Finance	80000

3. Union (U)

- ☆ Any relation is a **set**.
- ☆ Similar to **union** operation in Set Theory.
- ☆ It is a **binary** operator.
- ☆ It is a set of all objects that are a member of A, or B, or both.
- ☆ Like project, the **duplicate rows** are eliminated.
- ☆ It is denoted by U.
- ☆ Syntax: $\Pi_{\text{Column}}(\text{Relation}_1) \cup \Pi_{\text{Column}}(\text{Relation}_2)$

3. Union (U)

R	Alphabets
	A
	B
	C
	E
	F

S	Characters
	A
	B
	S
	E
	F
	l
	#

R U S	R U S
	A
	B
	C
	E
	F
	\$
	l
	#

Union operator is denoted by $\cup$ symbol and it is used to select all the rows (tuples) from two tables (relations).

Lets say we have two relations R1 and R2 both have same columns and we want to select all the tuples(rows) from these relations then we can apply the union operator on these relations.

Note: The rows (tuples) that are present in both the tables will only appear once in the union set. In short you can say that there are no duplicates present after the union operation.

3. Union (U)

Example 1: List all customer names associated with the bank either as an account holder or a loan borrower.

DEPOSITOR	Cu_Name	Acc_No	BORROWER	Cu_Name	Loan_No
R	Tom	A-101	S	John	L-201
	Amy	A-502		Smith	L-658
	Rose	A-304		Rose	L-254
	John	A-689		Jack	L-547

R U S	Cu_Name
	Tom
	Amy
	Rose
	John
	Smith
	Jack

Solution: $\Pi_{Cu_Name}(Depositor) \cup \Pi_{Cu_Name}(Borrower)$

Example – University Database

Instructor (ID, Name, Dept_Name, Salary)

Course (Course_ID, Title, Dept_Name, Credits)

Department (Dept_Name, Building, Budget)

Section (Course_ID, Sec_ID, Semester, Year, Building, Room_No, Time_slot_ID)

Teaches (ID, Course_ID, Sec_ID, Semester, Year)

Student (ID, Name, Dept_Name, Tot_Cred)

Advisor (S_ID, I_ID)

Takes (ID, Course_ID, Sec_ID, Semester, Year, Grade)

Classroom (Building, Room_Number, Capacity)

Time_Slot (Time_Slot_ID, Day, Start_Time, End_Time)

3. Union (U)

Example 2: Find the set of all courses taught in the Fall 2009 semester, the Spring 2010 semester, or both.

Solution:

To find the set of all courses taught in the Fall 2009 semester, we write:

$$\Pi_{\text{Course_Id}} (\sigma_{\text{Semester} = \text{'Fall'} \wedge \text{year} = 2009} (\text{SECTION}))$$

To find the set of all courses taught in the Spring 2010 semester, we write:

$$\Pi_{\text{Course_Id}} (\sigma_{\text{Semester} = \text{'Spring'} \wedge \text{year} = 2010} (\text{SECTION}))$$

To answer the query, we need the union of these two sets:

$$\Pi_{\text{Course_Id}} (\sigma_{\text{Semester} = \text{'Fall'} \wedge \text{year} = 2009} (\text{SECTION})) \cup \Pi_{\text{Course_Id}} (\sigma_{\text{Semester} = \text{'Spring'} \wedge \text{year} = 2010} (\text{SECTION}))$$

Two important conditions

For $R \cup S$ to be valid,

1. R and S must be of **same arity**.
2. For all i ,

Domain of the i^{th} attribute of R = **Domain** of i^{th} attribute of S.

Homework

Question 1: List all the course IDs which are taken in Spring 2020 or Fall 2021.

Question 2: List all the instructors' ID who taught courses in Spring 2020 or Fall 2021.

Instructor (ID, Name, Dept_Name, Salary)

Course (Course_ID, Title, Dept_Name, Credits)

Department (Dept_Name, Building, Budget)

Section (Course_ID, Sec_ID, Semester, Year, Building, Room_No, Time_slot_ID)

Teaches (ID, Course_ID, Sec_ID, Semester, Year)

Student (ID, Name, Dept_Name, Tot_Cred)

Advisor (S_ID, I_ID)

Takes (ID, Course_ID, Sec_ID, Semester, Year, Grade)

Classroom (Building, Room_Number, Capacity)

Time_Slot (Time_Slot_ID, Day, Start_Time, End_Time)

4. Set Difference (-)

- ☆ It is like **the same set difference** in Set Theory.
- ☆ It is a **binary** operation.
- ☆ To find tuples that are in one relation but are **not in another**.
- ☆ $R - S = \text{Tuples in } R \text{ but not in } S.$
- ☆ It is denoted by minus (-) symbol.

<https://www.youtube.com/watch?v=dS214kONAZA>

4. Set Difference (-)

R	Alphabets	S	Characters
	A		A
	B		B
	C		S
	D		L
	E		Y
	F		l
			#

R - S

R - S

C

D

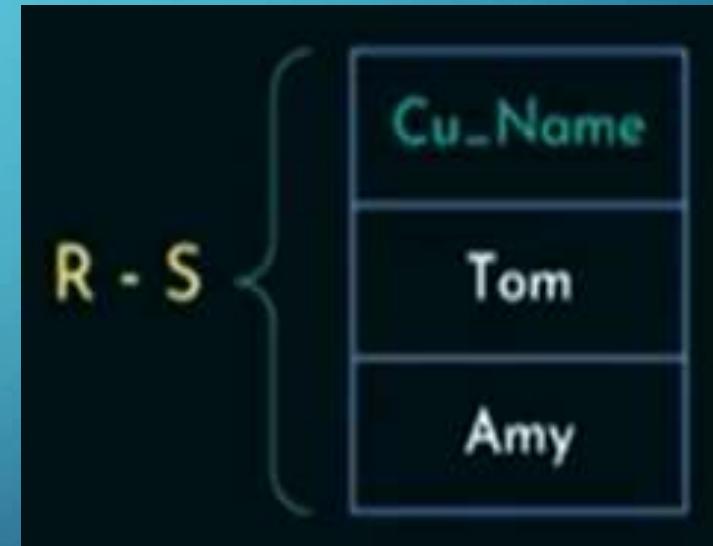
E

F

4. Set Difference (-)

Example 1: List all customer names those who have a deposit account but not availed loan.

DEPOSITOR	Cu_Name	Acc_No	BORROWER	Cu_Name	Loan_No
	Tom	A-101		John	L-201
	Amy	A-502		Smith	L-658
	Rose	A-304		Rose	L-254
	John	A-689		Jack	L-547



Solution: $\Pi_{Cu_Name} (Depositor) - \Pi_{Cu_Name} (Borrower)$

4. Set Difference (-)

Example 2: Find all the courses taught in the Fall 2009 semester but not in Spring 2010.

Solution:

To find the set of all courses taught in the Fall 2009 semester, we write:

$$\Pi_{\text{Course_id}} (\sigma_{\text{Semester} = \text{'Fall'} \wedge \text{year} = 2009} (\text{SECTION}))$$

To find the set of all courses taught in the Spring 2010 semester, we write:

$$\Pi_{\text{Course_id}} (\sigma_{\text{Semester} = \text{'Spring'} \wedge \text{year} = 2010} (\text{SECTION}))$$

To answer the query, we need the minus of these two sets:

$$\Pi_{\text{Course_id}} (\sigma_{\text{Semester} = \text{'Fall'} \wedge \text{year} = 2009} (\text{SECTION})) - \Pi_{\text{Course_id}} (\sigma_{\text{Semester} = \text{'Spring'} \wedge \text{year} = 2010} (\text{SECTION}))$$

Homework1

R	Alphabets
	A
	B
	C
	D
	E
	F

S	Characters
	A
	B
	S
	L
	Y
	1
	#

S - R

Homework2

Question 1: List all the instructors' ID who are not advisors.

Question 2: List all the instructors' ID who taught in Spring 2020 but not advisors.

Instructor (ID, Name, Dept_Name, Salary)

Course (Course_ID, Title, Dept_Name, Credits)

Department (Dept_Name, Building, Budget)

Section (Course_ID, Sec_ID, Semester, Year, Building, Room_No, Time_slot_ID)

Teaches (ID, Course_ID, Sec_ID, Semester, Year)

Student (ID, Name, Dept_Name, Tot_Cred)

Advisor (S_ID, I_ID)

Takes (ID, Course_ID, Sec_ID, Semester, Year, Grade)

Classroom (Building, Room_Number, Capacity)

Time_Slot (Time_Slot_ID, Day, Start_Time, End_Time)

5. Cartesian Product (X)

- ☆ Cartesian product associates **every** tuple of R_1 with **every** tuple of R_2 .
- ☆ It is a **binary** operation.
- ☆ It is denoted by cross (X) symbol.
- ☆ $R_1 \times R_2 =$ All possible pairing.
- ☆ Same attribute may appear in R_1 and R_2 .
- ☆ $R = \text{Depositor} \times \text{Borrower}$.

5. Cartesian Product (X)

Example: Perform Depositor X Borrower.

DEPOSITOR	
Cu_Name	Acc_No
Tom	A-101
Rose	A-304

BORROWER	
Cu_Name	Loan_No
John	L-201
Smith	L-658
Rose	L-254
Jack	L-547

DEPOSITOR X BORROWER			
Cu_Name	Acc_No	Cu_Name	Loan_No
Tom	A-101	John	L-201
Tom	A-101	Smith	L-658
Tom	A-101	Rose	L-254
Tom	A-101	Jack	L-547
Rose	A-304	John	L-201
Rose	A-304	Smith	L-658
Rose	A-304	Rose	L-254
Rose	A-304	Jack	L-547

Example No.02

Q: Find the customer name who has taken loan from "Pune" branch.

borrower relation

<u>cname</u>	<u>lno</u>
Asish	L1
Rupa	L2
Sima	L2

loan relation

<u>lno</u>	<u>bname</u>	<u>amt</u>
L1	Pune	4,00,000
L2	Noida	6,00,000

<u>Cname</u>	<u>borrower Lno</u>	<u>Loan Lno</u>	<u>bname</u>	<u>amt</u>
Asish	L1	L1	Pune	4,00,000
Asish	L1	L2	Noida	6,00,000
Rupa	L2	L1	Pune	4,00,000
Rupa	L2	L2	Noida	6,00,000
Sima	L2	L1	Pune	4,00,000
Sima	L2	L2	Noida	6,00,000

- Account(ano, bname, bal)
- Branch(bname, bcity, assets)
- Customer(cname, cstreet, ccity)
- Loan(lno, bname, amt)
- Depositor(cname, ano)
- Borrower(cname, lno)

$\pi_{cname} (\sigma_{\text{borrower.lno} = \text{loan.lno} \wedge \text{bname} = \text{"Pune"}} (\text{borrower} \times \text{loan}))$

Rename Operation(ρ) IT IS USED TO
GIVE THE NAME OF '**OUR CHOICE**' TO A NEW
'**RELATION**' OBTAINED FROM AFTER USING
ANY RELATIONAL ALGEBRA OPERATION.

IT IS AN UNARY OPERATION.

$\rho(R, E)$

R:- NEW RELATION NAME THAT WE WANT

E:- RELATIONAL ALGEBRA OPERATION EXPRESSION

IT IS ALSO USED TO RENAME ATTRIBUTE.

RELATIONAL ALGEBRA OPERATIONS

STUDENT (R)

S_ID	S_Name	S_Age	S_State	S_Course
1	Deepak	25	Delhi	B.Com
2	Rahul	24	Pune	B.Sc
3	Kapil	30	Delhi	B.com
4	Sudesh	22	Banglore	MSc.
5	Rahul	29	Pune	B.com

NEW NAME- Student-details

Retrieve age and state from student relation and store result in Student-detail

$\rho(\text{Student-details}, \pi_{\langle \text{S.Age}, \text{S.State} \rangle}(\text{STUDENT}))$

Rename variables S-Age and S-State to Age and State respectively

$\rho(\text{Student-details}(\text{Age}, \text{State}), \pi_{\langle \text{S.Age}, \text{S.State} \rangle}(\text{Student}))$

Relational Algebra Operations

Additional operations:

- ☆ Set Intersection ($\cap$)
- ☆ Assignment Operation ($=$)
- ☆ Natural Join ($\bowtie_*$)
- ☆ Division Operation ($\div$)
- ☆ Outer Join
 - Left Outer Join ($\bowtie_{\text{L}}$)
 - Right Outer Join ($\bowtie_{\text{R}}$)
 - Full Outer Join ($\bowtie_{\text{F}}$)

Set Intersection ($\cap$)

- ☆ Any relation is a **set**.
- ☆ Similar to **intersection** operation in Set Theory.
- ☆ It is a **binary** operator.
- ☆ It is a set of all objects that are a member of both A and B.
- ☆ It is denoted by $\cap$.
- ☆ Syntax: $\Pi_{\text{Column}}(\text{Relation_1}) \cap \Pi_{\text{Column}}(\text{Relation_2})$

Set Intersection ($\cap$)

R	Alphabets
	A
	B
	C
	D
	E
	F

S	Characters
	A
	B
	S
	E
	F
	!
	#

$R \cap S$	$R \cap S$
	A
	B
	E
	F

Set Intersection ($\cap$)

Example 1: Find the names of all customers who have deposited money and also availed loan.

Solution: $\Pi_{Cu_Name}(\text{Depositor}) \cap \Pi_{Cu_Name}(\text{Borrower})$

DEPOSITOR	Cu_Name	Acc_No
	Tom	A-101
	Amy	A-502
	Rose	A-304
	John	A-689

R

BORROWER	Cu_Name	Loan_No
	John	L-201
	Smith	L-658
	Rose	L-254
	Jack	L-547

S

$R \cap S$

Cu_Name
Rose
John

Example – University Database

Instructor (ID, Name, Dept_Name, Salary)

Course (Course_ID, Title, Dept_Name, Credits)

Department (Dept_Name, Building, Budget)

Section (Course_ID, Sec_ID, Semester, Year, Building, Room_No, Time_slot_ID)

Teaches (ID, Course_ID, Sec_ID, Semester, Year)

Student (ID, Name, Dept_Name, Tot_Cred)

Advisor (S_ID, I_ID)

Takes (ID, Course_ID, Sec_ID, Semester, Year, Grade)

Classroom (Building, Room_Number, Capacity)

Set Intersection ($\cap$)

Example 2: Find the set of all courses taught in the Fall 2009 semester and the Spring 2010 semesters.

Solution:

To find the set of all courses taught in the Fall 2009 semester, we write:

$$\Pi_{\text{Course_id}} (\sigma_{\text{Semester} = \text{'Fall'} \wedge \text{year} = 2009} (\text{SECTION}))$$

To find the set of all courses taught in the Spring 2010 semester, we write:

$$\Pi_{\text{Course_id}} (\sigma_{\text{Semester} = \text{'Spring'} \wedge \text{year} = 2010} (\text{SECTION}))$$

To answer the query, we need the intersection of these two sets:

$$\Pi_{\text{Course_id}} (\sigma_{\text{Semester} = \text{'Fall'} \wedge \text{year} = 2009} (\text{SECTION})) \cap \Pi_{\text{Course_id}} (\sigma_{\text{Semester} = \text{'Spring'} \wedge \text{year} = 2010} (\text{SECTION}))$$

Assignment Operator

- The **assignment operation** ($\leftarrow$) provides a convenient way to express complex queries.
 - It writes query as a sequential program consisting of:
 - a series of **assignments**
 - followed by an expression whose value is displayed as a **result** of the query.
 - **Assignment** must always be made to a **temporary relation variable**.

Example:

▣ **Division operation** $A \div B = \Pi_X(A) - \Pi_X((\Pi_X(A) \times B) - A)$

▣ Write $A \div B$ as

$\text{temp1} \leftarrow \Pi_X(A)$

$\text{temp2} \leftarrow \Pi_X((\text{temp1} \times B) - A)$

$\text{result} \leftarrow \text{temp1} - \text{temp2}$

- ▣ The result to the right of the $\leftarrow$ is assigned to the relation variable on the left of the $\leftarrow$
- ▣ May use variable in subsequent expressions.

Assignment Operator – Example No.02

Assignment Operation in Relational Algebra

Q: Find all customers who have account but no loan

$$r_1 \leftarrow \pi_{\text{cname}}(\text{depositor})$$

$$r_2 \leftarrow \pi_{\text{cname}}(\text{borrower})$$

$$\text{result} = r_1 - r_2$$

- Account(ano, bname, bal)
- Branch(bname, bcity, assets)
- Customer(cname, cstreet, ccity)
- Loan(lno, bname, amt)
- Depositor(cname, ano)
- Borrower(cname, lno)

Types of JOINS

1. Inner Join (Join):

- ▣ Theta join
- ▣ Equi join
- ▣ Natural join

2. Outer join:

(Extension of join)

- ▣ Left Outer Join
- ▣ Right Outer Join
- ▣ Full Outer Join

Types of JOINS

1. Inner Join:

- Contains only those tuples that satisfy the matching condition

- **Theta(θ) join**
- **Equi join**
- **Natural join**

2. Outer join:

- Extension of join
- Contains matching tuples that satisfy the matching condition, along with some or all tuples that do not satisfy the matching condition
- Contains all rows from either one or both relation

- **Left Outer Join**
- **Right Outer Join**
- **Full Outer Join**

Types of JOINS

1. Inner Join:

- Contains only those tuples that satisfy the matching condition

▣ Theta(θ) / Conditional Join

- ▣ $A \bowtie_{\theta} B$
- ▣ uses all kinds of comparison operators(<, >, <=, >=, =, ≠)

▣ Equi Join

- ▣ Special case of theta join
- ▣ uses only equality (=) comparison operator

▣ Natural join

- ▣ $A \bowtie B$
- ▣ Based on common attributes in both relation
- ▣ does not use any comparison operator

2. Outer join:

- Contains matching tuples that satisfy the matching condition, along with some or all tuples that do not satisfy the matching condition

- Contains all rows from either one or both relation

▣ Left Outer Join

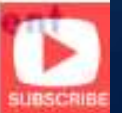
- ▣ Left relation tuples will always be in result whether the value is matched or not

▣ Right Outer Join

- ▣ Right relation tuples will always be in result whether the value is matched or not

▣ Full Outer Join

- ▣ Tuples from both relations are present in result, whether the value is matched or not



Natural Join

- ❑ **Natural join** can only be performed *if there is at least one common attribute* (column) that exist between two relations. In addition, the **attributes** must have the **same name and domain**.
- ❑ In **Natural Join** resulting relation the **common attributes appears only once**.
- ❑ **Notation:** $A \bowtie B$ (Natural join does not use any comparison operator)
- ❑ The **result of the natural join** is the set of all combinations of tuples in two relations **A** and **B** that are equal on their common attribute names.
- ❑ The **Natural Join** of two relations can be **obtained** by applying a **projection operation to equi join** of two relations. And in terms of basic operators:

Natural Join = Cartesian product + Selection + Projection

Example: Natural Join

Courses

CID	Course	Dept
CS01	Database	CS
ME01	Mechanics	ME
EE01	Electronics	EE

HOD

Dept	Head
CS	Rohan
ME	Sara
EE	Jiya

Courses ⋈ HOD

CID	Course	Dept	Head
CS01	Database	CS	Rohan
ME01	Mechanics	ME	Sara
EE01	Electronics	EE	Jiya

Equivalent to: $\Pi_{CID, Course, Courses.Dept, Head} (\sigma_{Courses1.Dept = HOD.Dept}(Courses \times HOD))$

Query 1:

- Find the names of all customers who have a loan at the bank, along with the loan number and the loan amount.

- Using Cartesian Product:

$\Pi_{\text{customer-name, loan. loan-number, amount}} (\sigma_{\text{borrower. loan-number} = \text{loan. loan-number}} (\text{borrower} \times \text{loan}))$

- Using Natural Join:

$\Pi_{\text{customer-name, loan.loan-number, amount}} (\text{borrower} \bowtie \text{loan})$

```
branch (branch-name, branch-city, assets)
customer (customer-name, customer-street, customer-city)
account (account-number, branch-name, balance)
loan (loan-number, branch-name, amount)
depositor (customer-name, account-number)
borrower (customer-name, loan-number)
```


Emp		Dept	
Eno	Ename	DepNo	Eno
1	Ram	D ₁	1
1	Ram	D ₂	2
1	Ram	D ₃	4
2	Vasun	D ₁	1
2	Vasun	D ₂	2
2	Vasun	D ₃	4
3	Ravi	D ₁	1
3	Ravi	D ₂	2
3	Ravi	D ₃	4
4	Amrit	D ₁	1
4	Amrit	D ₂	2
4	Amrit	D ₃	4

'Natural JOIN'
 "find the Emp Names who is working in a department"
Join = Cross Product + Condition

Emp	ENo	E-name	Address
	1	Ram	Delhi
	2	Vasun	Chd
	3	Ravi	Chd
	4	Amrit	Delhi

Dept	DepNo	Name	Eno
	D ₁	HR	1
	D ₂	IT	2
	D ₃	MRKT	4

Select E-name from emp where address = "Delhi"

**Select E-name from Emp,Dept
 where Emp.E-No = Dept.E-No**

1	Ram	Delhi	D ₁	Delhi	1
1	Ram	Delhi	D ₂	Pune	2
1	Ram	Delhi	D ₃	Patna	4
2	Varun	Chd	D ₁	Delhi	1
2	Varun	Chd	D ₂	Pune	2
2	Varun	Chd	D ₃	Patna	4
3	Ravi	Chd	D ₁	Delhi	1
3	Ravi	Chd	D ₂	Pune	2
3	Ravi	Chd	D ₃	Patna	4
4	Amrit	Delhi	D ₁	Delhi	1
4	Amrit	Delhi	D ₂	Pune	2
4	Amrit	Delhi	D ₃	Patna	4

'Equi JOIN' =
find the Emp name who worked in a department having location same as their address?

Emp	E-No	E-name	Address
	1	Ram	Delhi
	2	Varun	Chd
	3	Ravi	Chd
	4	Amrit	Delhi

DepNo	Location	E-no	<u>Dept</u>
D ₁	Delhi	1	
D ₂	Pune	2	
D ₃	Patna	4	

Select E-Name from Emp, Dept
where Emp.E-No = Dept.E-No
AND Emp.Address = Dept.Address

Π E-Name (σ Emp.E-No = Dept.E-No $\wedge$ Emp.Address = Dept.Address) (Emp $\bowtie$ Dept)

Outer Joins

Theta Join, Equijoin, and Natural Join are called inner joins. An inner join includes only those tuples with matching attributes and the rest are discarded in the resulting relation. Therefore, we need to use outer joins to include all the tuples from the participating relations in the resulting relation. There are three kinds of outer joins – left outer join, right outer join, and full outer join.

Left Outer Join($R \bowtie S$)

All the tuples from the Left relation, R, are included in the resulting relation. If there are tuples in R without any matching tuple in the Right relation S, then the S-attributes of the resulting relation are made NULL.

Left	
A	B
100	Database
101	Mechanics
102	Electronics
Right	
A	B
100	Alex
102	Maya
104	Mira

Courses $\bowtie$ HoD			
A	B	C	D
100	Database	100	Alex
101	Mechanics	---	---
102	Electronics	102	Maya

Left	
A	B
100	Database
101	Mechanics
102	Electronics
Right	
A	B
100	Alex
102	Maya
104	Mira

Right Outer Join: (R $\bowtie$ S)

All the tuples from the Right relation, S, are included in the resulting relation. If there are tuples in S without any matching tuple in R, then the R-attributes of resulting relation are made NULL.

Courses $\bowtie$ HoD			
A	B	C	D
100	Database	100	Alex
102	Electronics	102	Maya
---	---	104	Mira

Left	
A	B
100	Database
101	Mechanics
102	Electronics
Right	
A	B
100	Alex
102	Maya
104	Mira

Full Outer Join: (R ⋈ S)

All the tuples from both participating relations are included in the resulting relation. If there are no matching tuples for both relations, their respective unmatched attributes are made NULL.

Courses ⋈ HoD			
A	B	C	D
100	Database	100	Alex
101	Mechanics	---	---
102	Electronics	102	Maya
---	---	104	Mira

'Equi JOIN' =
find the Emp name who worked in a department having location
Same as their address?

Emp	E.No	E_name	Address
	1	Ram	Delhi
	2	Vasna	Chd
	3	Ravi	Chd
	4	Amit	Delhi

DeptNo	Location	E.No
D ₁	Delhi	1
D ₂	Pune	2
D ₃	Patna	4

Dept

Select E-Name from Emp, Dept
where Emp.E-No = Dept.E-No
AND Emp.Address = Dept.Address

□ E-Name (σ Emp.E-No = Dept.E-No $\wedge$ Emp.Address = Dept.Address) (Emp ⋈ Dept)